PRÁCTICA N.º 9 — ENUNCIADO

Despliegue en Kubernetes con Helm y GitOps

De la imagen publicada en la Práctica 8 a un clúster que se autorrepara y se sincroniza desde Git

Asignatura	COM-600 · Microservicios · Gestión 2/2026
Tema del programa	Tema 7 — Kubernetes, Estrategias de Despliegue y GitOps
Modalidad	Parte 1 guiada en laboratorio · Parte 2 individual
Duración estimada	Parte 1: 6 horas · Parte 2: trabajo autónomo
Puntaje	100 puntos (40 + 60)
Carpeta de trabajo	practica9-kubernetes

Nota: Este documento es el ENUNCIADO: contiene los comandos, las explicaciones y los ejercicios. Las capturas y las respuestas NO se escriben aquí, sino en el documento complementario «Hoja de Evidencias», que es el que se entrega.

1. Objetivos

Al finalizar esta práctica el estudiante será capaz de:

- Levantar un clúster de Kubernetes local con k3d y verificar que responde.
- Desplegar en ese clúster la imagen que la Práctica 8 publicó en el registro de contenedores.
- Distinguir el enfoque imperativo del declarativo y justificar por qué el segundo es el que se usa.
- Escribir manifiestos de Deployment, Service e Ingress y aplicarlos con kubectl apply.
- Separar la configuración y los secretos del código mediante ConfigMap y Secret.
- Configurar sondas de readiness y liveness y comprobar su efecto en una actualización progresiva.
- Realizar un rolling update sin corte de servicio y revertirlo con rollout undo.
- Empaquetar la aplicación como un chart de Helm y desplegar el mismo chart con dos juegos de valores.
- Operar un flujo GitOps con Argo CD: sincronizar desde el repositorio y corregir un desvío del clúster.

2. Qué necesita saber antes de empezar

El contenido conceptual de esta práctica se desarrolla en **Tema 7 — Kubernetes, Estrategias de Despliegue y GitOps**. Aquí solo se recuerda lo mínimo necesario para ejecutar los laboratorios.

- **Pod:** la unidad más pequeña que Kubernetes ejecuta: uno o varios contenedores que comparten red y almacenamiento. Es efímero y desechable.
- **Deployment:** declara cuántas réplicas de un pod deben existir y con qué imagen; el clúster trabaja sin descanso para que la realidad coincida con esa declaración.
- **Service:** nombre y dirección estables para un conjunto de pods que cambian de IP todo el tiempo. Reparte el tráfico entre los que están listos.
- **Ingress:** regla de entrada HTTP que publica un Service hacia afuera del clúster por nombre de host y ruta. Necesita un controlador que la ejecute.
- **Manifiesto declarativo:** archivo YAML que describe el estado deseado, no los pasos para llegar a él. Se versiona en Git y se aplica con kubectl apply.
- **Chart:** paquete de Helm: los manifiestos convertidos en plantillas más un values.yaml con los valores que cambian entre entornos.

Repaso: La teoría —arquitectura del plano de control, planificador, controladores, bucle de reconciliación, estrategias de despliegue azul-verde y canario, y los principios de GitOps— está en las diapositivas del Tema 7. Esta práctica es la ejecución de todo eso sobre un clúster real.

3. Requisitos previos

Requisito	Verificación	Observación
Docker Desktop o Engine 24+	docker ps	k3d ejecuta los nodos del clúster como contenedores de Docker
k3d 5.6 o superior	k3d version	Es la herramienta que usa esta guía. Minikube 1.32+ sirve como alternativa: los manifiestos son los mismos y solo cambian los comandos de creación del clúster y de acceso por Ingress
kubectl 1.29 o superior	kubectl version --client	Cliente oficial del clúster. Debe ser como máximo una versión menor por detrás del servidor
Helm 3.14 o superior	helm version	Necesario a partir del Laboratorio 6
Imagen de la Práctica 8	docker pull ghcr.io/USUARIO/tareas-api:1.0.0	Es la imagen que el pipeline de la Práctica 8 publicó en el registro. Si el paquete es privado, hágalo público o cree un Secret de tipo docker-registry
Cuenta de GitHub	Acceso al repositorio de la materia	El Laboratorio 7 necesita un repositorio público con la carpeta de manifiestos
Memoria libre	4 GB disponibles	El clúster y Argo CD conviven sin problema en 4 GB; con menos, cierre el navegador y otros contenedores
Git	git --version	Para versionar los manifiestos y subir la entrega

Atención: Esta práctica continúa directamente la Práctica 8. Allí el pipeline construyó la imagen del servicio y la publicó en el registro; aquí esa misma imagen se despliega en un clúster. Trabaje con la etiqueta exacta que su pipeline generó, no con «latest».

Cree una carpeta de trabajo llamada practica9-kubernetes y realice TODO el trabajo dentro de ella. Esa carpeta se sube al repositorio al final.

Los comandos se muestran precedidos por el símbolo $. NO escriba el símbolo $ al copiar el comando.

Antes de cada comando destructivo verifique en qué clúster está parado con kubectl config current-context. Borrar recursos en el contexto equivocado es el accidente más frecuente de esta materia.

Un Secret de Kubernetes NO cifra nada: solo codifica en base64. Nunca suba a Git un Secret con valores reales.

4. Estructura y forma de entrega

La práctica se apoya en dos documentos:

Documento	Para qué sirve	¿Se entrega?
Enunciado (este archivo)	Guía de trabajo: comandos, explicaciones, ejercicios y rúbrica. Se consulta mientras se trabaja.	No
Hoja de Evidencias	Formulario donde se pegan las 20 capturas y se responden las 8 preguntas de comprobación de la Parte 1.	Sí, en PDF

El trabajo tiene dos partes obligatorias:

Parte	Contenido	Puntaje
PARTE 1 — Guiada	8 laboratorios paso a paso. Se ejecutan los comandos y se documenta cada resultado en la Hoja de Evidencias.	40 pts
PARTE 2 — Ejercicios propuestos	6 ejercicios de resolución individual. Se entregan como código en el repositorio más un informe en PDF.	60 pts

Parte	Contenido	Puntaje
TOTAL		100 pts

Entregables

- Hoja de Evidencias en PDF, con las capturas pegadas y las preguntas de comprobación respondidas (Parte 1).
- Repositorio Git público con la carpeta practica9-kubernetes: la carpeta k8s/ con los manifiestos, la carpeta chart/ con el chart de Helm, los archivos de valores por entorno, el manifiesto de la Application de Argo CD y un README.md con las instrucciones para reproducir el despliegue desde cero.
- Informe PDF de la Parte 2 con la resolución de cada ejercicio, capturas de la evidencia y las respuestas a las preguntas de reflexión.

PARTE 1 — PRÁCTICA GUIADA

Ejecute cada comando en el orden indicado. Cuando aparezca un aviso de «Captura N.º», tome la captura de pantalla y péguela en el recuadro correspondiente de la Hoja de Evidencias. Las «Pregunta N.º» se responden en esa misma hoja, con sus propias palabras: son las que demuestran que entendió lo que ejecutó.

> **Consejo:** Trabaje con los dos documentos abiertos a la vez: este enunciado para leer y ejecutar, y la Hoja de Evidencias para ir registrando. Las capturas y las preguntas están numeradas igual en ambos.

Laboratorio 0 — Verificación del entorno y creación del clúster

Un clúster de Kubernetes en la máquina de un estudiante es perfectamente posible: k3d levanta k3s dentro de contenedores de Docker y en menos de un minuto se tiene un plano de control funcionando. Antes de crear nada, confirme que las herramientas están instaladas.

Paso 1. Verifique las versiones instaladas

Abra una terminal en la carpeta de trabajo y ejecute:

```
$ docker --version
$ k3d version
$ kubectl version --client
$ helm version
$ docker ps
```

El último comando debe responder con una tabla, aunque esté vacía. Si responde con un error de conexión, Docker no está iniciado y nada de lo que sigue funcionará.

> **Captura N.º 1:** salida de los cinco comandos anteriores en su terminal, con las versiones visibles. → *péguela en la Hoja de Evidencias.*

Paso 2. Cree el clúster

Un solo comando crea el servidor de control, dos nodos de trabajo y publica el puerto 80 del balanceador hacia su máquina. Ese último detalle es el que permitirá abrir la aplicación en el navegador en el Laboratorio 3.

```
$ mkdir practica9-kubernetes && cd practica9-kubernetes
$ k3d cluster create com600 --agents 2 -p "8080:80@loadbalancer"
# --agents 2 crea dos nodos de trabajo
# -p publica el puerto 80 del clúster en el 8080 de su máquina
```

Si usa Minikube: El equivalente es minikube start --nodes 2 y después minikube addons enable ingress. El resto de la práctica es idéntico salvo la forma de acceder por Ingress, que se indica en el Laboratorio 3.

Paso 3. Confirme que el clúster responde

kubectl habla con el clúster a través de un contexto guardado en ~/.kube/config. k3d lo crea y lo deja seleccionado automáticamente.

```
$ kubectl config current-context
$ kubectl get nodes -o wide
$ kubectl get pods -A
```

Debe ver tres nodos en estado Ready y, en el espacio de nombres kube-system, los pods de CoreDNS, del balanceador y de Traefik, que es el controlador de Ingress que k3s trae incorporado.

Captura N.º 2: salida de kubectl get nodes con los tres nodos en Ready y el nombre del contexto actual. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 1: Los tres nodos que acaba de ver son en realidad contenedores de Docker corriendo en su máquina. ¿Qué diferencia hay entre este clúster y uno de producción, y qué cosas de esta práctica seguirían siendo válidas allá? → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Laboratorio 1 — El despliegue imperativo y por qué no sirve

Kubernetes permite crear recursos escribiendo comandos, sin ningún archivo. Funciona, es rápido y es exactamente lo que no vamos a hacer durante el resto de la práctica. Conviene verlo una vez para entender qué se pierde.

Paso 1. Cree un despliegue y expóngalo con comandos sueltos

Use la imagen que su pipeline de la Práctica 8 publicó en el registro. Reemplace USUARIO por su nombre de usuario de GitHub y la etiqueta por la que corresponda.

```
$ kubectl create deployment tareas --image=ghcr.io/USUARIO/tareas-api:1.0.0
$ kubectl expose deployment tareas --port=80 --target-port=3000
$ kubectl get deployment,service,pods
```

En pocos segundos debe haber un pod en Running y un Service con una IP interna del clúster. Si el pod queda en ImagePullBackOff, revise el anexo de errores: casi siempre es un nombre de imagen mal escrito o un paquete privado.

Captura N.º 3: salida de kubectl get deployment,service,pods mostrando el pod en Running. → *péguela en la Hoja de Evidencias.*

Paso 2. Compruebe que la aplicación responde dentro del clúster

Todavía no hay ninguna puerta hacia afuera. kubectl port-forward abre un túnel temporal entre su máquina y el Service, útil solo para diagnosticar.

```
# terminal 1
$ kubectl port-forward service/tareas 8081:80

# terminal 2
$ curl http://localhost:8081/tareas
```

Qué es y qué no es: port-forward es una herramienta de diagnóstico, no una forma de publicar un servicio: vive mientras el comando esté abierto y solo sirve a la máquina que lo ejecutó.

Paso 3. Borre todo y observe qué quedó

Borre los dos recursos y pregúntese qué le queda en el disco para volver a crearlos exactamente igual.

```
$ kubectl delete service tareas
$ kubectl delete deployment tareas
$ kubectl get all
$ ls
```

La carpeta está vacía. Todo lo que existió durante los últimos minutos vivía únicamente en la memoria del clúster y en su historial de comandos.

Captura N.º 4: salida de kubectl get all después del borrado y de ls mostrando que no quedó ningún archivo. *→ péguela en la Hoja de Evidencias.*

Pregunta N.º 2: Suponga que el clúster se pierde y hay que reconstruirlo tal como estaba. Con el enfoque imperativo, ¿de dónde saldría la información necesaria? Nombre dos consecuencias concretas de no tener el estado deseado escrito en un archivo versionado. *→ respóndala en la Hoja de Evidencias, con sus propias palabras.*

Laboratorio 2 — Manifiestos declarativos: Deployment y Service

Ahora el mismo resultado, pero escrito. Un manifiesto declara el estado deseado; Kubernetes se encarga de alcanzarlo y de mantenerlo. Ese archivo es el que se versiona, se revisa y, en el Laboratorio 7, se convierte en la única fuente de verdad.

Paso 1. Escriba el Deployment

Cree la carpeta k8s/ y dentro el archivo deployment.yaml. Note que no dice cómo crear los pods: dice cuántos deben existir y con qué imagen.

k8s/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: tareas
spec:
  replicas: 2
  selector:
    matchLabels:
      app: tareas
  template:
    metadata:
      labels:
        app: tareas
    spec:
      containers:
        - name: api
          image: ghcr.io/USUARIO/tareas-api:1.0.0
          ports: [{ containerPort: 3000 }]
```

La pieza que más confunde: selector.matchLabels y template.metadata.labels deben coincidir. El Deployment reconoce como suyos los pods que llevan esa etiqueta; si no coinciden, el clúster acepta el manifiesto y no crea nada útil.

Paso 2. Aplique el manifiesto y observe el resultado

kubectl apply compara lo que usted declara con lo que existe y hace lo necesario para que coincidan. Ejecutarlo dos veces seguidas no duplica nada.

```
$ kubectl apply -f k8s/deployment.yaml
$ kubectl get deployment tareas
$ kubectl get pods -l app=tareas -o wide
$ kubectl describe deployment tareas | head -30
```

En get pods debe ver dos pods, posiblemente repartidos en nodos distintos. Fíjese en la columna NODE: el planificador decidió dónde ponerlos sin que usted interviniera.

Captura N.º 5: salida de kubectl get pods -l app=tareas -o wide con las dos réplicas en Running y el nodo de cada una. → *péguela en la Hoja de Evidencias.*

Paso 3. Lea los registros y demuestre que el pod es desechable

Vea la salida del contenedor y después borre un pod a mano. No lo reemplace usted: observe quién lo hace.

```
$ kubectl logs -l app=tareas --tail=20
$ kubectl delete pod -l app=tareas --field-selector=status.phase=Running \
    --wait=false | head -1
$ kubectl get pods -l app=tareas -w
```

El controlador del Deployment nota que hay menos réplicas de las declaradas y crea las que faltan en segundos. Corte la observación con Ctrl+C.

Esto es el bucle de reconciliación: Nadie ordenó crear un pod nuevo. El controlador compara el estado deseado con el real y actúa. Es la misma idea que después aplicará Argo CD, pero a nivel de todo el repositorio.

Paso 4. Escriba el Service y verifique sus endpoints

Los pods cambian de IP cada vez que nacen. El Service da un nombre estable y reparte el tráfico entre los pods que estén listos.

k8s/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: tareas
spec:
  selector:
    app: tareas
  ports:
    - port: 80
      targetPort: 3000
```

Aplíquelo y revise sus endpoints: son las IP de los pods que el Service está sirviendo en este momento. Un Service sin endpoints es un Service que no encontró ningún pod con esa etiqueta.

Captura N.º 6: salida de kubectl apply del Service seguida de kubectl get endpoints tareas mostrando dos direcciones. → *péguela en la Hoja de Evidencias.*

Paso 5. Escale las réplicas de forma declarativa

Cambie replicas de 2 a 4 en k8s/deployment.yaml, guarde y vuelva a aplicar. Después vuelva a 2 del mismo modo.

```
$ kubectl apply -f k8s/deployment.yaml
$ kubectl get pods -l app=tareas
$ kubectl get endpoints tareas
```

Los endpoints del Service crecen y se reducen solos: nadie tuvo que registrar las direcciones nuevas en ningún lado.

Captura N.º 7: las cuatro réplicas en Running y la lista de endpoints del Service con cuatro direcciones. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 3: Compare este laboratorio con el anterior. ¿Qué puede hacer ahora que antes era imposible, en términos de revisión por un compañero, de reconstrucción del entorno y de auditoría de quién cambió qué? → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Laboratorio 3 — Publicar hacia afuera con Ingress

El Service resuelve el tráfico dentro del clúster. Para que un navegador de su máquina llegue a la aplicación hace falta una regla de entrada y un controlador que la ejecute. k3s trae Traefik ya instalado, así que solo hay que escribir la regla.

Paso 1. Confirme que existe un controlador de Ingress

Sin controlador, un Ingress es un archivo que nadie lee: se acepta y no ocurre nada.

```
$ kubectl get pods -n kube-system | grep traefik
$ kubectl get ingressclass
```

Debe aparecer el pod de Traefik en Running y una IngressClass llamada traefik marcada como predeterminada.

Paso 2. Escriba la regla de entrada

Cree k8s/ingress.yaml. La regla dice: todo lo que llegue al host tareas.localhost por la ruta raíz va al Service tareas en el puerto 80.

k8s/ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: tareas
spec:
  rules:
    - host: tareas.localhost
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
```

```
            service:
              name: tareas
              port:
                number: 80
```

Aplíquelo con kubectl apply -f k8s/ingress.yaml y compruébelo con kubectl get ingress.

Captura N.º 8: salida de kubectl get ingress mostrando el host tareas.localhost y la clase traefik. → *péguela en la Hoja de Evidencias.*

Paso 3. Abra la aplicación en el navegador

Recuerde que al crear el clúster se publicó el puerto 80 del balanceador en el 8080 de su máquina. El dominio .localhost resuelve a 127.0.0.1 en los navegadores modernos sin tocar el archivo hosts.

```
$ curl -H "Host: tareas.localhost" http://localhost:8080/tareas
# y en el navegador:  http://tareas.localhost:8080/tareas

# con Minikube el acceso es distinto:
$ minikube ip   # agregue esa IP y el host a su archivo hosts
```

Captura N.º 9: el navegador mostrando la respuesta de la API en http://tareas.localhost:8080, con la barra de direcciones visible. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 4: Recorra el camino completo de esa petición: desde el navegador hasta el proceso que la atiende, nombrando cada pieza que interviene. ¿Qué ocurriría si el Service quedara sin endpoints mientras el Ingress sigue existiendo? → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Laboratorio 4 — Configuración y secretos fuera de la imagen

La misma imagen debe poder correr en laboratorio y en producción. Lo único que cambia es la configuración, y por eso la configuración no puede vivir dentro de la imagen. Kubernetes ofrece dos objetos para eso: ConfigMap para lo que puede leer cualquiera y Secret para lo que no.

Paso 1. Declare la configuración no sensible

Cree k8s/configmap.yaml con los valores que cambian entre entornos y que no comprometen nada si se leen.

```
k8s/configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: tareas-config
data:
  APP_ENTORNO: "laboratorio"
  APP_PAGINA_TAM: "20"
```

Aplíquelo con kubectl apply -f k8s/configmap.yaml.

Paso 2. Cree el Secret sin escribirlo en un archivo

Este Secret se crea con un comando, a propósito: así el valor real nunca llega al repositorio. En el Laboratorio 7 verá cómo se resuelve esto de forma definitiva.

```
$ kubectl create secret generic tareas-secret \
```

```
    --from-literal=APP_CLAVE_API=clave-de-laboratorio-2026
$ kubectl get secret tareas-secret -o yaml | head -8
```

Observe que el valor aparece como una cadena ilegible. Esa ilegibilidad engaña a mucha gente.

Captura N.º 10: salida de kubectl get secret -o yaml con el valor codificado visible. → *péguela en la Hoja de Evidencias.*

Paso 3. Monte ambos como variables de entorno

Agregue el bloque envFrom al contenedor en k8s/deployment.yaml, justo debajo de ports, y vuelva a aplicar el manifiesto.

```
k8s/deployment.yaml (fragmento del contenedor)
      containers:
        - name: api
          image: ghcr.io/USUARIO/tareas-api:1.0.0
          ports: [{ containerPort: 3000 }]
          envFrom:
            - configMapRef: { name: tareas-config }
            - secretRef: { name: tareas-secret }
```

Aplique de nuevo y verifique dentro del pod. Note que el Deployment crea pods nuevos: las variables de entorno se fijan al arrancar el contenedor, no se actualizan en caliente.

Captura N.º 11: salida de kubectl exec en un pod ejecutando env y mostrando las tres variables APP_. → *péguela en la Hoja de Evidencias.*

Paso 4. Demuestre que base64 no es cifrado

Cualquiera con permiso de lectura sobre el Secret recupera el valor original en un solo comando. No hay clave, no hay algoritmo, no hay nada que romper.

```
$ kubectl get secret tareas-secret \
    -o jsonpath="{.data.APP_CLAVE_API}" | base64 -d; echo
# en PowerShell:
# [Text.Encoding]::UTF8.GetString([Convert]::FromBase64String("<valor>"))
```

Captura N.º 12: la clave en texto plano recuperada desde el Secret con base64 -d. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 5: Acaba de recuperar el secreto en texto plano sin ninguna credencial especial. Entonces, ¿qué problema resuelve realmente un Secret frente a escribir la clave en el Deployment, y qué haría falta agregar para que el valor esté verdaderamente protegido? → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Laboratorio 5 — Sondas de salud y actualización progresiva

Un pod puede estar corriendo y no ser capaz de atender: todavía está arrancando, perdió la conexión con la base o se quedó bloqueado. Kubernetes no lo adivina; hay que decírselo con sondas. Sin readiness, el rolling update manda tráfico a pods que no están listos y el corte es visible.

Paso 1. Agregue las dos sondas al contenedor

Añada este bloque debajo de envFrom en k8s/deployment.yaml. Ambas consultan el mismo endpoint de salud, pero sirven a propósitos distintos.

```
k8s/deployment.yaml (fragmento del contenedor)
          readinessProbe:
            httpGet: { path: /salud, port: 3000 }
            initialDelaySeconds: 3
            periodSeconds: 5
          livenessProbe:
            httpGet: { path: /salud, port: 3000 }
            initialDelaySeconds: 15
            periodSeconds: 10
```

La diferencia que hay que entender: Si falla la readiness, el pod sale de los endpoints del Service y deja de recibir tráfico, pero sigue vivo. Si falla la liveness, el contenedor se reinicia. Confundirlas produce reinicios en cadena durante un arranque lento.

Paso 2. Aplique y observe el despliegue de la versión 2

Aplique el manifiesto con las sondas, espere a que termine y luego cambie la etiqueta de la imagen a 2.0.0 y vuelva a aplicar. Observe el rollout en vivo.

```
$ kubectl apply -f k8s/deployment.yaml
$ kubectl rollout status deployment/tareas
# cambie la etiqueta de la imagen a 2.0.0 y aplique otra vez
$ kubectl apply -f k8s/deployment.yaml
$ kubectl get pods -l app=tareas -w
```

Verá pods nuevos en ContainerCreating mientras los viejos siguen en Running. Solo cuando un pod nuevo pasa la readiness, Kubernetes retira uno viejo. Corte con Ctrl+C.

Captura N.º 13: la lista de pods durante el rolling update, con réplicas de las dos versiones conviviendo. → *péguela en la Hoja de Evidencias.*

Paso 3. Compruebe que no hubo corte de servicio

Repita el despliegue mientras un bucle golpea la aplicación sin parar. Es la única forma honesta de afirmar que no hubo corte.

```
# terminal 1: bucle de peticiones
$ while true; do curl -s -o /dev/null -w "%{http_code} " \
    -H "Host: tareas.localhost" http://localhost:8080/salud; sleep 0.3; done

# terminal 2: vuelva a la imagen 1.0.0 y aplique
$ kubectl apply -f k8s/deployment.yaml
```

La terminal 1 debe mostrar una fila continua de códigos 200. Si aparecen 502 o 503, el pod nuevo recibió tráfico antes de estar listo: revise la sonda de readiness.

Captura N.º 14: las dos terminales: el bucle con códigos 200 continuos y el rollout ejecutándose al mismo tiempo. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 6: Explique con sus palabras qué habría pasado en ese bucle si el Deployment no tuviera readinessProbe. ¿En qué momento exacto habrían aparecido los errores y por qué? → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Paso 4. Consulte el historial y revierta

Kubernetes guarda las revisiones del Deployment. Revertir no es volver a desplegar la versión anterior a mano: es una operación de un solo comando.

```
$ kubectl rollout history deployment/tareas
$ kubectl rollout undo deployment/tareas
$ kubectl rollout status deployment/tareas
$ kubectl describe deployment tareas | grep -i image
```

La imagen vuelve a la de la revisión anterior. Recuerde que el archivo YAML de su disco NO cambió: el clúster y el repositorio quedaron desalineados, y eso es precisamente el problema que resuelve el Laboratorio 7.

> **Captura N.º 15:** el historial de revisiones y la salida de rollout undo con la imagen restaurada. → *péguela en la Hoja de Evidencias.*

Laboratorio 6 — Empaquetar la aplicación con Helm

Tiene cuatro manifiestos y una sola diferencia real entre laboratorio y producción: el número de réplicas, la etiqueta de la imagen y el host. Duplicar la carpeta entera para cambiar tres valores es la forma más segura de que ambos entornos se separen con el tiempo. Helm convierte los manifiestos en plantillas y deja los valores afuera.

Paso 1. Cree la estructura del chart

Helm genera un chart de ejemplo con muchos archivos. Para esta práctica lo dejaremos mínimo: borre las plantillas generadas y escriba las suyas.

```
$ helm create chart
$ rm -rf chart/templates/* chart/charts
$ ls chart
```

Deben quedar Chart.yaml, values.yaml y la carpeta templates/ vacía.

Paso 2. Escriba los valores y la plantilla del Deployment

Reemplace el contenido de chart/values.yaml por lo siguiente. Este archivo es el contrato del chart: lo único que un operador necesita conocer para desplegarlo.

```
chart/values.yaml
replicaCount: 2
image:
  repository: ghcr.io/USUARIO/tareas-api
  tag: "1.0.0"
service:
  port: 80
ingress:
  host: tareas.localhost
```

Paso 3. Convierta el Deployment en plantilla

Cree chart/templates/deployment.yaml. Es el mismo manifiesto del Laboratorio 2 con tres valores sustituidos por expresiones.

```
chart/templates/deployment.yaml
apiVersion: apps/v1
```

```
kind: Deployment
metadata:
  name: {{ .Release.Name }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: {{ .Release.Name }}
  template:
    metadata:
      labels:
        app: {{ .Release.Name }}
    spec:
      containers:
        - name: api
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          ports: [{ containerPort: 3000 }]
```

Haga lo mismo con service.yaml e ingress.yaml del Laboratorio 2, sustituyendo el nombre por {{ .Release.Name }}, el puerto por {{ .Values.service.port }} y el host por {{ .Values.ingress.host }}.

Paso 4. Revise el resultado antes de instalar y luego instale

helm template renderiza las plantillas sin tocar el clúster. Revisar esa salida antes de instalar es un hábito que evita sorpresas.

```
$ helm template tareas ./chart | head -25
$ kubectl delete -f k8s/   # retire lo del laboratorio anterior
$ helm install tareas ./chart
$ helm list
$ kubectl get pods -l app=tareas
```

Captura N.º 16: salida de helm install seguida de helm list con la versión 1 desplegada y los pods en Running. → *péguela en la Hoja de Evidencias.*

Paso 5. Actualice, revierta y despliegue un segundo entorno

Cree chart/values-prod.yaml con los valores del otro entorno y observe que el mismo chart produce dos instalaciones distintas y aisladas.

```
chart/values-prod.yaml
replicaCount: 4
image:
  tag: "2.0.0"
ingress:
  host: tareas-prod.localhost
```

Ejecute: helm upgrade tareas ./chart --set replicaCount=3, después helm history tareas y helm rollback tareas 1. Finalmente instale el segundo entorno con helm install tareas-prod ./chart -f chart/values-prod.yaml y compare ambos con helm list.

Captura N.º 17: salida de helm history con las tres revisiones y de helm list con las dos instalaciones del mismo chart. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 7: El clúster ya sabía revertir un Deployment con rollout undo. ¿Qué revierte helm rollback que kubectl rollout undo no puede revertir? Piense en un cambio que afecte al Service y al Ingress a la vez. → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Laboratorio 7 — GitOps con Argo CD

Hasta aquí, quien despliega es usted desde su terminal. Eso significa que el clúster puede alejarse del repositorio sin que nadie se entere, como ya ocurrió con el rollout undo del Laboratorio 5. GitOps invierte la dirección: el repositorio declara el estado deseado y un agente dentro del clúster trabaja permanentemente para que la realidad coincida.

Paso 1. Instale Argo CD en el clúster

Argo CD se instala como una aplicación más, en su propio espacio de nombres.

```
$ kubectl create namespace argocd
$ kubectl apply -n argocd -f \
    https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
$ kubectl get pods -n argocd -w
```

Espere a que todos los pods pasen a Running: la primera vez tarda varios minutos porque descarga las imágenes. Corte con Ctrl+C cuando estén todos listos.

Paso 2. Entre a la interfaz de Argo CD

La contraseña inicial del usuario admin queda guardada en un Secret. Ya sabe cómo leerlo.

```
$ kubectl -n argocd get secret argocd-initial-admin-secret \
    -o jsonpath="{.data.password}" | base64 -d; echo
$ kubectl port-forward svc/argocd-server -n argocd 8090:443
# abra https://localhost:8090 y acepte el aviso del certificado
```

Paso 3. Suba los manifiestos a un repositorio y declare la Application

Suba la carpeta k8s/ a un repositorio público de GitHub. Después cree el archivo argocd/application.yaml, que le dice a Argo CD qué repositorio vigilar y dónde aplicarlo.

```
argocd/application.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: tareas
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/USUARIO/practica9-kubernetes.git
    targetRevision: main
    path: k8s
  destination:
    server: https://kubernetes.default.svc
    namespace: default
  syncPolicy:
    automated: { prune: true, selfHeal: true }
```

Aplíquelo con kubectl apply -f argocd/application.yaml. En pocos segundos la aplicación aparece en la interfaz con su árbol de recursos.

Captura N.º 18: la interfaz de Argo CD con la aplicación tareas en estado Synced y Healthy, mostrando el árbol de recursos. → *péguela en la Hoja de Evidencias.*

Paso 4. Cambie el repositorio y observe la sincronización

Antes de desinstalar el chart, retire la instalación de Helm para que no compita por los mismos recursos: helm uninstall tareas y helm uninstall tareas-prod. Después modifique replicas en k8s/deployment.yaml, confirme el cambio en Git y súbalo.

```
$ git add k8s/ && git commit -m "Escalar a 3 réplicas"
$ git push
$ kubectl get pods -l app=tareas -w
# o pulse Refresh en la interfaz para no esperar el sondeo
```

Sin que usted ejecute ningún kubectl apply, el clúster llega a tres réplicas. El único acto de despliegue fue un git push.

Captura N.º 19: el commit en GitHub y, al lado, los pods del clúster reflejando el nuevo número de réplicas. *→ péguela en la Hoja de Evidencias.*

Paso 5. Provoque un desvío y observe la corrección automática

Ahora haga lo que hace todo el mundo un viernes por la tarde: cambiar algo directamente en el clúster, sin pasar por el repositorio.

```
$ kubectl scale deployment tareas --replicas=7
$ kubectl get pods -l app=tareas -w
# observe la interfaz de Argo CD durante los siguientes segundos
```

Argo CD marca la aplicación como OutOfSync y, como la política automated tiene selfHeal activo, devuelve el clúster al número de réplicas declarado en Git. El cambio manual dura lo que tarda el siguiente ciclo de reconciliación.

Captura N.º 20: la interfaz de Argo CD mostrando el estado OutOfSync y, a continuación, el retorno a Synced con las réplicas del repositorio. *→ péguela en la Hoja de Evidencias.*

Pregunta N.º 8: Un compañero afirma que GitOps sirve para desplegar más rápido. Argumente por qué esa no es su ventaja principal y nombre dos problemas concretos de operación que sí resuelve. ¿Qué le habría pasado al rollout undo del Laboratorio 5 en este esquema? *→ respóndala en la Hoja de Evidencias, con sus propias palabras.*

Con esto el circuito completo de la materia queda cerrado: el código se prueba (Práctica 7), el pipeline lo construye y publica la imagen (Práctica 8) y el clúster la despliega y la mantiene sincronizada con el repositorio (esta práctica). Al terminar, borre el clúster con k3d cluster delete com600 si necesita liberar memoria; los manifiestos quedan en Git y recrear todo es cuestión de dos comandos.

PARTE 2 — EJERCICIOS PROPUESTOS

Esta parte se resuelve de forma individual. Cada ejercicio se entrega con: (a) el código o los archivos de configuración en el repositorio, (b) las capturas que demuestren su funcionamiento y (c) la explicación pedida, en el informe PDF.

Nota: Los ejercicios se resuelven sobre su proyecto integrador, no sobre la API de tareas del laboratorio. Del 1 al 4 son obligatorios para todos; el 5 y el 6 son los que separan una entrega correcta de una sobresaliente.

Ejercicio 1. Manifiestos de un servicio de su proyecto (10 pts)

Tome uno de los microservicios de su proyecto integrador cuya imagen ya esté publicada en el registro por el pipeline de la Práctica 8 y llévelo al clúster de forma completamente declarativa.

Se pide:

- Carpeta k8s/ con Deployment, Service, Ingress y ConfigMap, todos aplicables con un solo kubectl apply -f k8s/.
- Al menos dos réplicas y una etiqueta de imagen concreta: el uso de latest invalida el ejercicio.
- Límites de recursos (requests y limits de CPU y memoria) razonados en el informe.
- Captura de la aplicación respondiendo en el navegador a través del Ingress.

Criterio de aceptación: *Borrar el espacio de nombres y volver a aplicar la carpeta debe reconstruir el servicio completo sin ningún comando adicional.*

Ejercicio 2. Sondas que reflejan la salud real del servicio (10 pts)

La mayoría de los servicios responde 200 en /salud aunque la base de datos esté caída. Corrija eso en su proyecto y demuéstrelo.

Se pide:

- Un endpoint de readiness que verifique de verdad las dependencias (base de datos, cola o servicio del que dependa) y otro de liveness que solo confirme que el proceso responde.
- Las dos sondas configuradas en el Deployment con tiempos justificados en el informe.
- Una demostración: detenga la dependencia y capture cómo el pod sale de los endpoints del Service sin ser reiniciado.
- Una explicación de qué habría ocurrido si esa verificación estuviera en la liveness en lugar de la readiness.

Criterio de aceptación: *Debe verse la captura del pod en estado Running pero con READY 0/1, y el Service sin ese endpoint.*

Ejercicio 3. Actualización progresiva sin corte, medida (10 pts)

Demuestre con datos, no con afirmaciones, que su servicio se actualiza sin perder ni una petición.

Se pide:

- Dos versiones de la imagen de su servicio, publicadas en el registro con etiquetas distintas.
- Un script o bucle que envíe peticiones continuas durante todo el despliegue y cuente cuántas respondieron 200 y cuántas no.
- La estrategia de rolling update configurada explícitamente (maxSurge y maxUnavailable) y justificada.
- La misma medición hecha sin readinessProbe, para comparar los dos resultados en el informe.

Criterio de aceptación: *El informe debe presentar los dos conteos, con y sin sonda de readiness, y explicar la diferencia.*

Ejercicio 4. Chart de Helm con dos entornos (12 pts)

Empaquete su servicio como un chart y demuestre que un mismo chart sostiene dos entornos sin duplicar un solo manifiesto.

Se pide:

- Chart propio con Chart.yaml, values.yaml y las plantillas de todos sus recursos.
- Dos archivos de valores, values-dev.yaml y values-prod.yaml, que difieran al menos en réplicas, etiqueta de imagen, host y recursos.
- Las dos instalaciones conviviendo en el clúster, en espacios de nombres distintos, con la captura de helm list.

- Una demostración de helm upgrade seguido de helm rollback, con la salida de helm history.
- La salida de helm template revisada en el informe: qué produjo cada juego de valores.

Criterio de aceptación: *Ningún manifiesto puede estar duplicado entre entornos. Si hay dos copias de la misma plantilla, el ejercicio no acredita.*

Ejercicio 5. GitOps sobre su proyecto (8 pts)

Deje su proyecto gobernado por Argo CD: que el repositorio, y no su terminal, sea la única fuente de verdad del clúster.

Se pide:

- Un repositorio de manifiestos separado o una carpeta claramente delimitada, con la Application de Argo CD versionada.
- Sincronización automática con selfHeal y prune activos.
- Evidencia de un despliegue completo cuyo único acto humano fue un commit y un push.
- Evidencia de un desvío provocado a mano y de su corrección automática, con las capturas de OutOfSync y de Synced.

Criterio de aceptación: *Durante todo el ejercicio no debe aparecer ningún kubectl apply de sus manifiestos: si lo hay, no es GitOps.*

Ejercicio 6. Manejo honesto de los secretos y reconstrucción desde cero (10 pts)

Los Secret en base64 no pueden subirse a Git, pero GitOps exige que todo esté en Git. Resuelva esa contradicción en su proyecto y demuestre que el entorno se reconstruye completo.

Se pide:

- Una solución de secretos que permita versionar el archivo sin exponer el valor: Sealed Secrets, SOPS con age, un gestor externo o el mecanismo que elija, documentado y funcionando.
- Una justificación de la solución elegida frente a al menos otra alternativa.
- Un README.md que permita a otra persona, en una máquina limpia, reconstruir el clúster y el despliegue completo desde cero.
- La reconstrucción efectivamente probada: borre el clúster con k3d cluster delete, siga su propio README y capture el resultado final funcionando.

Criterio de aceptación: *Se evaluará borrando el clúster y siguiendo su README al pie de la letra. Cada paso que usted deba explicar de viva voz es un paso que falta en el documento.*

Rúbrica de evaluación

Parte 1 — Práctica guiada (40 puntos)

Criterio	Descripción	Pts
Capturas completas	Las 20 capturas están pegadas en la Hoja de Evidencias, son legibles y corresponden a la máquina del estudiante (se ve su usuario o el nombre del equipo).	15
Ejecución correcta	Las salidas muestran el resultado esperado, sin errores sin resolver.	10
Preguntas de comprobación	Las 8 preguntas están respondidas con criterio propio, no copiadas del enunciado.	10
Orden y presentación	Hoja de Evidencias completa, con datos del estudiante y declaración firmada, entregada en PDF y dentro del plazo.	5
Subtotal		40

Parte 2 — Ejercicios propuestos (60 puntos)

Ejercicio	Qué se evalúa	Pts
1. Manifiestos de un servicio de su proyecto	Cuatro manifiestos correctos y aplicables de una vez, etiqueta fija, recursos declarados y justificados, evidencia de acceso externo.	10
2. Sondas que reflejan la salud real del servicio	Distinción real entre readiness y liveness, tiempos justificados, demostración con la dependencia caída, análisis del caso contrario.	10
3. Actualización progresiva sin corte, medida	Dos versiones desplegadas, medición cuantitativa del corte, parámetros de la estrategia justificados, comparación con y sin sonda.	10
4. Chart de Helm con dos entornos	Chart completo y plantillado, dos entornos sin duplicación, instalaciones simultáneas, ciclo upgrade-rollback documentado.	12
5. GitOps sobre su proyecto	Application versionada, sincronización automática funcionando, despliegue por commit, desvío corregido con evidencia.	8
6. Manejo honesto de los secretos y reconstrucción desde cero	Secretos versionables sin exposición, comparación de alternativas, README reproducible y prueba real de reconstrucción desde cero.	10
Subtotal		60

Penalizaciones

Situación	Descuento
Uso de la etiqueta latest en cualquier manifiesto entregado	−5 pts
Recursos creados con comandos imperativos en lugar de manifiestos versionados	−5 pts
Secret con valores reales subido al repositorio en texto plano o en base64	−8 pts
Deployment sin readinessProbe en los ejercicios que la exigen	−4 pts
Manifiestos duplicados entre entornos en lugar de un chart con dos juegos de valores	−4 pts
Capturas que no corresponden a la máquina del estudiante	Anulación de la Parte 1
Entrega fuera de plazo	Según reglamento de la materia

Anexo — Errores frecuentes y cómo resolverlos

Mensaje o síntoma	Causa probable	Solución
ImagePullBackOff o ErrImagePull	El nombre o la etiqueta de la imagen están mal escritos, o el paquete del registro es privado.	Verifique con docker pull la misma cadena del manifiesto. Si el paquete es privado, hágalo público o cree un Secret de tipo docker-registry y refiéralo en imagePullSecrets.
CrashLoopBackOff	El contenedor arranca y termina de inmediato: falta una variable de entorno, no encuentra la base de datos o el proceso sale con error.	Lea kubectl logs <pod> --previous, que muestra la salida del intento anterior. Corrija la causa antes de reaplicar.
El pod queda en Pending para siempre	Ningún nodo puede alojarlo: los requests de CPU o memoria superan lo disponible en el clúster local.	Revise kubectl describe pod <pod> en la sección Events. Baje los requests o libere memoria en su máquina.
El Service existe pero no responde y no tiene endpoints	El selector del Service no coincide con las etiquetas de los pods, o ningún pod pasa la readiness.	Compare kubectl get endpoints <svc> con kubectl get pods --show-labels y ajuste el selector o la sonda.
El Ingress se creó pero el navegador da 404 o no conecta	No hay controlador de Ingress, o el puerto del balanceador no está publicado hacia la máquina.	Confirme el pod de Traefik en kube-system y que el clúster se creó con -p "8080:80@loadbalancer". Con Minikube, habilite el complemento ingress.

Mensaje o síntoma	Causa probable	Solución
kubectl responde sobre recursos que usted no creó, o dice not found	Está apuntando a otro clúster o a otro espacio de nombres.	Ejecute kubectl config current-context y kubectl config get-contexts; cambie con kubectl config use-context k3d-com600 y agregue -n al comando si corresponde.
The connection to the server localhost:8080 was refused	kubectl no encuentra ninguna configuración de clúster: el clúster no existe o el archivo kubeconfig no está donde se espera.	Verifique con k3d cluster list y recree el kubeconfig con k3d kubeconfig merge com600 --kubeconfig-switch-context.
La aplicación de Argo CD queda en OutOfSync y no se corrige sola	La política de sincronización no es automática, o el repositorio, la rama o la ruta declarados no existen.	Revise syncPolicy.automated en la Application y compruebe repoURL, targetRevision y path. La pestaña App Details muestra el error exacto del repositorio.
Argo CD y Helm pelean por los mismos recursos	La instalación de Helm del Laboratorio 6 y la Application de Argo CD gestionan los mismos objetos.	Ejecute helm uninstall de la instalación anterior antes de dejar que Argo CD tome el control, o apúntelos a espacios de nombres distintos.

Antes de pedir ayuda: Antes de pedir ayuda ejecute estos tres comandos sobre el recurso que falla: kubectl get <recurso> -o wide, kubectl describe <recurso> —y lea la sección Events del final— y kubectl logs <pod>. Kubernetes casi siempre dice exactamente qué le falta; el problema es que hay que ir a leerlo.

Referencias

- Documentación oficial de Kubernetes — https://kubernetes.io/es/docs/concepts/
- k3d — https://k3d.io/
- Minikube — https://minikube.sigs.k8s.io/docs/
- Helm — https://helm.sh/docs/topics/charts/
- Argo CD — https://argo-cd.readthedocs.io/
- Principios de GitOps — https://opengitops.dev/
- Burns, Brendan y otros. Kubernetes: Up and Running, 3.ª edición. O'Reilly, 2022.
- Diapositivas del Tema 7 — Kubernetes, Estrategias de Despliegue y GitOps (COM-600).